2048-bit semiprime—that is, a number that is the product of exactly two primes. This resulting number has about 617 decimal digits. While it is relatively easy to generate such a semiprime, factoring it back into its original prime components is computationally infeasible with today's technology. Even the most powerful supercomputers would likely take hundreds of years to factor such a number using classical algorithms. This asymmetry is what makes RSA secure: multiplying two large primes is easy, whereas factoring the resulting semiprime back into its prime factors is extraordinarily difficult.

You can think of it like this: proving that unicorns exist is easy—you need to just show one. But proving that unicorns don't exist requires exhaustive evidence, which involves searching every possible corner of existence. Similarly, finding the product of two primes is easy if you already have them, but discovering them from their product is practically impossible without a breakthrough in mathematics or quantum computing.

Let us now see how SageMath can be used to generate two prime numbers, each of size 1024 bits. Keep in mind that this is not a trivial task, as generating large primes requires efficient algorithms and careful handling of randomness. Fortunately, SageMath is equipped with powerful number-theoretic tools that make this process straightforward. The SageMath program titled 'prime33.sage', shown below, accomplishes this task.

```
p = random_prime(2^1024 - 1, 2^1023)
print("1024-bit prime (in decimal):")
print(p)
print("\nNumber of decimal digits:", len(str(p)))
```

In the program, the first line calls the function *random_prime( )* to generate a random prime number with exactly 1024 bits. This is achieved by setting the upper bound to $2^{1024}-1$ and the lower bound to 2^{1023}, ensuring the resulting prime has exactly 1024 bits. The function *random_prime( )* in SageMath relies on mathematical libraries like FLINT (Fast Library for Number Theory) and GMP (GNU Multiple Precision Arithmetic Library) for high-performance large integer arithmetic and primality testing. The program is executed twice to generate the two required prime numbers. The output of each execution is shown in Figure 5. Notice that each 1024-bit prime number generated contains 309 decimal digits.

Now, let us implement a simplified version of RSA using the two 1024-bit prime numbers generated by the SageMath program 'prime33.sage'. The SageMath program titled 'rsa34.sage' (line numbers are included for clarity), shown below, provides a simplified yet secure RSA implementation. Keep in mind that when running this

```
# sage prime33.sage
1024-bit prime (in decimal):
12730412952429749927088002291925382711388155175125905756
61424583010212050363696761373046858526680205601437716912
26791988105630335992388397851499929520985166175131407750
06078107516979249072886225843762328718927918913059818260
10318566675450732258638471654793718780424259278068707110
70821648689901032419498393436641290

Number of decimal digits: 309
# sage prime33.sage
1024-bit prime (in decimal):
14965766542055491512974217543668287253152990560226048788
89389671996833007561646753879333082999307205906887428580
11144627518659477491011163228629612585768365667089885220
23876272492431180456950712003380870824717614266335451880
70644791056374293406544389534514045897451496224872918320
38878737828118324648558982460236190

Number of decimal digits: 309
```

Figure 5: Prime number generation with SageMath

program on your system, the variables *p* and *q* should be assigned the two prime numbers generated earlier. These assignment lines are omitted here to save space.

```
1.  n = p * q
2.  phi = (p - 1)*(q - 1)
3.  e = 65537
4.  d = inverse_mod(e, phi)
5.  m = 68
6.  c = power_mod(m, e, n)
7.  decrypted = power_mod(c, d, n)
8.  print("Original message:", m)
9.  print("Encrypted:", c)
10. print("Decrypted:", decrypted)
```

Now, let us try to understand the working of the program. Line 1 computes the RSA modulus *n*, the product of two large prime numbers *p* and *q*. It forms the public key along with the exponent *e*. The value of *n* determines the size of the RSA key and sets the range for encryption and decryption. Line 2 computes Euler's totient function $\varphi(n)$ (called *phi*), which is used in calculating the private exponent *d*. It counts the number of integers less than *n* that are coprime to *n*—that is, their greatest common divisor (gcd) with *n* is 1. Line 3 sets the public exponent *e*. A common choice for *e* is 65537 because it is a prime number large enough to ensure security while being small enough to allow efficient encryption. Line 4 computes the modular inverse of *e* modulo *phi*, storing the result in the variable *d*. This is the private exponent, and part of the private key. Line 5 assigns the original plain text message to the variable *m*. In this case, it is the number 68, which corresponds to the ASCII code for the character 'D'. Note that in real-world applications, *m* would be much larger or appropriately